

LINQ OrderByDescending Method in C# with Examples

In this article, I will discuss the **LINQ OrderByDescending Method in C#** with Examples. Please read our previous article discussing the [LINQ OrderBy Method in C#](#) with Examples. As part of this article, we will discuss the following pointers related to the LINQ OrderByDescending method.

1. [What is the LINQ OrderByDescending Method in C#?](#)
2. [Example of OrderByDescending Method using both Method and Query Syntax.](#)
3. [How do you use the OrderByDescending Method with Complex Type in C#?](#)
4. [How do you use the OrderByDescending method along with the Filtering method?](#)
5. [How do you create and use your own Comparer with the OrderByDescending Method in C#?](#)
6. [When should you use the Linq OrderByDescending Method in C#?](#)

What is the Linq OrderByDescending Method in C#?

The OrderByDescending method in LINQ is used to sort elements of a collection in descending order based on a specified key. It's part of the System.Linq namespace in .NET, works similarly to the OrderBy method but in reverse order. You can use OrderByDescending with any collection that implements the IEnumerable<T> interface, such as lists, arrays, or other enumerable types.

The point that you need to remember is that the OrderByDescending method will not change the data. It is just changing the order of the data. There are two overloaded versions of the OrderByDescending Method available inside LINQ. They are as follows.

```
public static IEnumerable<TSource> OrderByDescending<TSource, TKey>(this IEnumerable<TSource> source, Func<TSource, TKey> keySelector);  
  
public static IEnumerable<TSource> OrderByDescending<TSource, TKey>(this IEnumerable<TSource> source, Func<TSource, TKey> keySelector, IComparer<TKey> comparer);
```

Here, TSource is the data source, and TKey is the property of the data source's items that you want to sort by in descending order. The only difference between these two overloaded versions is that the second version takes the IComparer parameter, which compares the keys when creating our custom comparer. Let us understand using the LINQ OrderByDescending method in C# using both query syntax and method syntax using both primitive and reference data types.

Like most LINQ methods, OrderByDescending uses deferred execution. The sorting doesn't happen immediately when you call OrderByDescending. Instead, it returns an IEnumerable<T>. The sorting is performed when you iterate over this enumerable, like using a for each loop or converting it to a list.

LINQ OrderByDescending Method with Value Data Type in C#

Let us see an example of how the LINQ OrderByDescending Method works with Primitive Data Types such as Integer using C#. In the below example, we have a collection of integer data. And we need to sort the data in descending order. We use the LINQ OrderByDescending method using Method and Query Syntax for this.

```
using System;  
using System.Collections.Generic;
```

```

using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> intList = new List<int>() { 10, 45, 35, 29, 100, 69, 58, 50 };
            Console.WriteLine("Before Sorting the Data: ");
            foreach (var item in intList)
            {
                Console.Write(item + " ");
            }
            //Sorting the data in Descending Order Using Method Syntax
            var MS = intList.OrderByDescending(num => num);
            //Sorting the data in Descending Order Using Query Syntax
            var QS = (from num in intList
                      orderby num descending
                      select num).ToList();
            Console.WriteLine();
            Console.WriteLine("After Sorting the Data in Descending Order: ");
            foreach (var item in QS)
            {
                Console.Write(item + " ");
            }
            Console.ReadKey();
        }
    }
}

```

Now, run the application, and you will get the following output.

```

Before Sorting the Data:
10 45 35 29 100 69 58 50
After Sorting the Data in Descending Order:
100 69 58 50 45 35 29 10

```

Note: The OrderByDescending method sorts the elements of the source sequence in place and returns an `IOrderedEnumerable<T>` that can be enumerated to retrieve the sorted elements. It's also often followed by `ThenByDescending` or `ThenBy` if you need to perform a secondary sort on the sequence.

LINQ OrderByDescending Method with String Data Type in C#

Let us see an example of how the LINQ OrderByDescending Method works with String Data Type using C#. In the below example, we have a collection of string data, i.e., a collection of names. We then sort the data in descending order using the LINQ OrderByDescending method with Method and Query syntax. In the Query Syntax, while we are sorting the data in descending order, the use of the descending operator is mandatory. If we omit the descending keyword, then by default, it will be ascending.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> stringList = new List<string>() { "Preety", "Tiwary",
"Priyanka", "Dewangan", "Hina", "Sharma"};
            Console.WriteLine("Before Sorting:");
            foreach (var item in stringList)
            {
                Console.Write(item + " ");
            }
            //Sorting in Descending Order Using Method Syntax
            var MS = stringList.OrderByDescending(name => name);
            //Sorting in Descending Order Using Query Syntax
            var QS = (from name in stringList
                      orderby name descending
                      select name).ToList();
            Console.WriteLine("\nAfter Sorting in Descending Order:");
            foreach (var item in MS)
            {
                Console.Write(item + " ");
            }
            Console.ReadKey();
        }
    }
}

```

When you run the above code, you will get the following output as expected.

```

Before Sorting:
Preety Tiwary Priyanka Dewangan Hina Sharma
After Sorting in Descending Order:
Tiwary Sharma Priyanka Preety Hina Dewangan

```

LINQ OrderBy Method with Complex Data Type in C#:

Let us see how the LINQ OrderByDescending Method works with Complex Data Types in C# with Examples. We are going to work with the following Student class. So, create a class file named **Student.cs** and copy and paste the following code. As you can see, we created the Student class with four properties: **ID**, **FirstName**, **LastName**, and **Brach**. We then created one method, i.e., **GetAllStudents()**, to return a list of students.

```

using System.Collections.Generic;
namespace LINQDemo

```

```

{
    public class Student
    {
        public int ID { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Branch { get; set; }
        public static List<Student> GetAllStudents()
        {
            List<Student> listStudents = new List<Student>()
            {
                new Student{ID= 101,FirstName = "Preety",LastName = "Tiwary",F
"CSE"}},
                new Student{ID= 102,FirstName = "Preety",LastName = "Agrawal",
"ETC"}},
                new Student{ID= 103,FirstName = "Priyanka",LastName = "Dewanga
"ETC"}},
                new Student{ID= 104,FirstName = "Hina",LastName = "Sharma",Bra
"ETC"}},
                new Student{ID= 105,FirstName = "Anugrag",LastName = "Mohanty"
"CSE"}},
                new Student{ID= 106,FirstName = "Anurag",LastName = "Sharma",F
"CSE"}},
                new Student{ID= 107,FirstName = "Pranaya",LastName = "Kumar",F
"CSE"}},
                new Student{ID= 108,FirstName = "Manoj",LastName = "Kumar",Bra
"ETC"}},
                new Student{ID= 109,FirstName = "Pranaya",LastName = "Rout",Br
"ETC"}},
                new Student{ID= 110,FirstName = "Saurav",LastName = "Rout",Bra
"CSE"}
            };
            return listStudents;
        }
    }
}

```

Now, we want to sort the student data based on the Branch of the Student in descending order. To do so, modify the Main method of the Program class as follows. In the code below, we are sorting the data in descending order based on the student branch using the LINQ OrderByDescending Method with both Method and Query Syntax.

```

using System.Linq;
using System;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Method Syntax

```

```

var MS = Student.GetAllStudents().OrderByDescending(x => x.Branch);
//Query Syntax
var QS = (from std in Student.GetAllStudents()
          orderby std.Branch descending
          select std);
foreach (var student in MS)
{
    Console.WriteLine(" Branch: " + student.Branch + ", Name : " +
student.FirstName + " " + student.LastName);
}

Console.ReadKey();
}
}
}

```

When you run the above code, you will get the following output.

```

Branch: ETC, Name :Preety Agrawal
Branch: ETC, Name :Priyanka Dewangan
Branch: ETC, Name :Hina Sharma
Branch: ETC, Name :Manoj Kumar
Branch: ETC, Name :Pranaya Rout
Branch: CSE, Name :Preety Tiwary
Branch: CSE, Name :Anugrag Mohanty
Branch: CSE, Name :Anurag Sharma
Branch: CSE, Name :Pranaya Kumar
Branch: CSE, Name :Saurav Rout

```

Linq OrderByDescending Method with Filtering Method

Now we need to fetch only the ETC branch students, and then we need to sort the students based on the first name in Ascending order. The most important point you need to remember is to use the Where Extension method before the OrderByDescending method. This is because it will first filter and then sort the filtered results, improving the application's performance. The following example uses LINQ OrderByDescending Method and LINQ Where Method with Query and Method Syntax.

```

using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Method Syntax
            var MS = Student.GetAllStudents()
                           .Where(std => std.Branch.ToUpper() == "ETC")
                           .OrderByDescending(x => x.FirstName).ToList();

            //Query Syntax

```

```

var QS = (from std in Student.GetAllStudents()
          where std.Branch.ToUpper() == "ETC"
          orderby std.FirstName descending
          select std);
foreach (var student in QS)
{
    Console.WriteLine(" Branch: " + student.Branch + ", Name : " +
student.FirstName + " " + student.LastName);
}
Console.ReadKey();
}
}
}

```

When you run the above code, you will get the following output as expected.

```

Branch: ETC, Name :Priyanka Dewangan
Branch: ETC, Name :Preety Agrawal
Branch: ETC, Name :Pranaya Rout
Branch: ETC, Name :Manoj Kumar
Branch: ETC, Name :Hina Sharma

```

How do you create and use your own Comparer with LINQ OrderByDescending Method in C#?

Creating and using our own Comparer with the LINQ OrderByDescending Method is also possible. We need to use the Second Overloaded version of the OrderByDescending Method, which takes the IComparer parameter. For a better understanding, please have a look at the below example.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class CaseInsensitiveComparer : IComparer<string>
    {
        public int Compare(string x, string y)
        {
            return string.Compare(x, y, true);
        }
    }
    public class MainClass
    {
        public static void Main()
        {
            CaseInsensitiveComparer caseInsensitiveComparer = new
CaseInsensitiveComparer();
            string[] Alphabets = { "a", "b", "c", "A", "B", "C" };

```

```

        var SortedAlphabets = Alphabets.OrderByDescending(aplhabet => aplhabet.Length,
        caseInsensitiveComparer);
        foreach (var item in SortedAlphabets)
        {
            Console.Write(item + " ");
        }
        Console.Read();
    }
}

```

Output: c C b B a A

When should you use the LINQ OrderByDescending Method in C#?

The LINQ OrderByDescending method is used when you need to sort a collection in descending order. This method is useful in various scenarios as follows:

- **Sorting in Reverse Order:** When you have a collection of simple types (like integers, strings, or dates) or complex objects and you want to sort them in descending order. For example, sorting a list of numbers from highest to lowest.
- **Sorting Complex Objects by a Specific Property:** If you're dealing with a collection of complex objects, OrderByDescending allows you to sort these objects based on a specific property. For instance, sorting a list of Employee objects by their Salary property in descending order to quickly find the highest earners.
- **Data Preparation for Display:** In user interfaces, you may need to display data in a sorted manner, and sometimes, this sorting needs to be in descending order, like showing the most recent orders or highest scores first.
- **Performance Optimization in Certain Scenarios:** In some cases, especially when dealing with large datasets, sorting the data in descending order before performing further operations like searches or additional filtering can be more efficient.
- **Use in LINQ Queries for Databases:** In LINQ to SQL or Entity Framework, using OrderByDescending translates to an SQL ORDER BY DESC clause, allowing the database to handle the sorting. This can be more efficient for large datasets.
- **Part of a Chain of LINQ Operations:** OrderByDescending is often used as part of a longer chain of LINQ operations, where the descending sort order is an integral part of complex data processing.
- **Secondary Sorting:** Sometimes, OrderByDescending is used after an OrderBy for secondary sorting. For example, you might first sort a list of products by category (using OrderBy) and then sort each category by price in descending order (using OrderByDescending).
- **Top-N Queries:** When you need to retrieve the top N elements with the highest values according to a specific criterion, you can use OrderByDescending to sort the data and then use methods like Take to limit the result set. Example: Retrieving the top 10 highest-scoring players in a game leaderboard.

In the next article, I will discuss [LINQ ThenBy and ThenByDescending Methods in C#](#) with Examples. I hope you enjoy this LINQ OrderByDescending Method with Examples article.